Data Structures (ETCCDS202)

Unit–1 Assignment: Foundations & Algorithmic Analysis

Name of the School: School of Engineering & Technology

Program : B.Tech (CSE / AIML)

Section : A

Student Name : Disha Saini

Roll No. : 2501730318

Submitted To : Ms. Neetu

Introduction

This assignment focuses on understanding recursion, algorithm efficiency, and abstract data types. The objective was to implement a Stack ADT and solve classical recursive problems such as Factorial, Fibonacci, Tower of Hanoi, and Binary Search. Along with implementation, time and space complexity analysis was performed to understand algorithm efficiency using Big-O notation.

Stack ADT

An Abstract Data Type (ADT) defines the behaviour of a data structure by specifying the operations that can be performed on it, without describing how those operations are implemented internally.

In this assignment, a Stack ADT was implemented using a Python class. The stack follows the LIFO (Last In, First Out) principle, meaning the last element inserted is the first one removed.

The following operations were implemented:

- push(x) - Inserts an element into the stack
- pop() - Removes and returns the top element
- peek() - Returns the top element without removing it
- is_empty() - Checks whether the stack is empty
- size() - Returns the number of elements in the stack

The stack was internally implemented using a Python list. Since Python list operations like append() and pop() from the end take constant time, all stack operations run in **O(1)** time complexity.

The Stack ADT was used to store and print the moves in the Tower of Hanoi problem, demonstrating practical usage of the abstract data type.

Factorial (Recursion + Complexity Analysis)

The factorial of a number n is defined as:

$$\boldsymbol{n! = n \times (n-1)!}$$

with the base case:

$$\boldsymbol{0! = 1}$$

In this assignment, the factorial function was implemented using recursion. The function calls itself with a smaller value of $n$until it reaches the base case.

Time Complexity

For each recursive call, the value of n decreases by 1. Therefore, if the input is n, the function makes **n** recursive calls.

- **(Big-O):** O(n)
- **Big-Theta (O):** O(n)
- **Big-Omega (Ω):** Ω(n)

Since the function always performs exactly n recursive calls (except for small base cases), the best, average, and worst cases are the same.

Space Complexity

Each recursive call is stored in the call stack until it returns. Therefore, the recursion depth is **n**.

- **Space Complexity:** O(n)

Fibonacci (Naive vs Memoized Recursion)

The Fibonacci sequence is defined as:

$$\boldsymbol{F(0) = 0, F(1) = 1}$$
$$\boldsymbol{F(n) = F(n-1) + F(n-2)}$$

Two versions of Fibonacci were implemented in this assignment: a naive recursive version and a memoized recursive version.

Naive Recursive Fibonacci

The naive recursive implementation directly follows the mathematical definition. However, it recalculates the same Fibonacci values multiple times.

For example, when calculating $F(5)$, the function computes $F(3)$and $F(2)$multiple times, leading to repeated work.

Time Complexity

The recurrence relation is approximately:

$$T(n) = T(n-1) + T(n-2)$$

This results in exponential growth.

- **(Big-O):** $O(2^n)$
- **Big-Theta (O):** $O(2^n)$
- **Big-Omega (Ω):** $\Omega(2^n)$

This exponential time complexity makes the naive version very slow for larger values of n.

Space Complexity

The recursion depth is at most n.

- **Space Complexity:** O(n)

<u>Memoized Fibonacci (Dynamic Programming)</u>

The memoized version stores previously computed Fibonacci values in a dictionary (memoization). This prevents repeated calculations of the same subproblems.

Each Fibonacci number is computed only once and stored for future use.

Time Complexity

Since each value from 0 to n is computed once:

- **(Big-O):** O(n)
- **Big-Theta (O):** O(n)
- **Big-Omega (Ω):** Ω(n)

This is a major improvement over the naive version.

Space Complexity

- Storage for memo dictionary - O(n)
- Recursion depth - O(n)

So overall:

- **Space Complexity:** O(n)

The call counter results clearly showed that the memoized version performs significantly fewer recursive calls compared to the naive version.

Tower of Hanoi (Recursion Analysis)

The Tower of Hanoi is a classic recursive problem where the goal is to move $n$disks from a source rod to a destination rod using an auxiliary rod, following these rules:

1. Only one disk can be moved at a time.
2. A larger disk cannot be placed on a smaller disk.

The recursive strategy used was:

- Move n -1 disks from source to auxiliary
- Move the nth disk from source to destination
- Move n-1 disks from auxiliary to destination

Number of Moves

The recurrence relation for the number of moves is:

$$\boldsymbol{T(n) = 2T(n-1) + 1}$$

Solving this recurrence gives:

$$\boldsymbol{T(n) = 2^n - 1}$$

This means the number of moves grows exponentially as $n$increases.

Time Complexity

Since the number of operations follows 2^n - 1:

- **(Big-O):** $O(2^n)$
- **Big-Theta (O):** $(2O^n)$
- **Big-Omega (Ω):** $\Omega(2^n)$

Space Complexity

The recursion depth is equal to n, because at most $n$function calls exist in the call stack at a time.

- **Space Complexity:** O(n)

In this assignment, the Stack ADT was used to store the sequence of moves before printing them, demonstrating the practical use of abstraction.

Recursive Binary Search (Divide & Conquer)

Binary Search is an efficient searching algorithm that works on a sorted array. It repeatedly divides the search space into two halves.

The steps are:

1. Find the middle element of the array.
2. If the key equals the middle element, return its index.
3. If the key is smaller, search the left half.
4. If the key is larger, search the right half.

This approach reduces the problem size by half in each recursive call.

Recurrence Relation

The recurrence for binary search is:

$$\boldsymbol{T(n) = T(n/2) + O(1)}$$

At each step, the problem size becomes half, and only constant work is done outside the recursive call.

Solving this recurrence gives:

$$\boldsymbol{T(n) = O(\log n)}$$

Time Complexity

- **Best Case:** O(1)
  (When the middle element is the key)
- **Average Case:** O(log n)
- **Worst Case:** O(log n)

Binary search is much faster than linear search for large datasets because the search space is reduced exponentially.

Space Complexity

Since the recursive version was implemented, the recursion depth is:

- **Space Complexity:** O(log n)

If implemented iteratively, the space complexity would be O(1).

Algorithm Paradigm Reflection

Binary Search - Divide & Conquer

Binary Search follows the Divide & Conquer paradigm because it divides the problem into smaller subproblems by splitting the array into two halves. It then recursively solves only one half of the problem. At each step, the size of the problem reduces to half, making the algorithm efficient with logarithmic time complexity.

Memoized Fibonacci - Dynamic Programming

The memoized Fibonacci implementation follows the Dynamic Programming paradigm. Dynamic Programming is used when a problem has overlapping subproblems and optimal substructure.

In the naive Fibonacci version, the same subproblems are solved multiple times. Memoization stores previously computed results and reuses them, preventing repeated computation. This reduces the time complexity from exponential $O(2^n)$ to linear $O(n)$.

Tower of Hanoi - Recursion

Tower of Hanoi is a pure recursive problem. It works by breaking the problem of moving n disks into smaller subproblems of moving $n - 1$ disks. The base case occurs when there is only one disk to move. The recursive structure clearly demonstrates how large problems can be solved using smaller similar problems.

Real-Life Examples of Paradigms

- **Divide & Conquer:** Searching for a word in a dictionary by opening it near the middle and deciding which half to search next.
- **Dynamic Programming:** Planning a road trip by remembering the shortest distances between cities instead of recalculating them every time.
- **Greedy Algorithm:** Giving change using the largest currency notes first to minimize the number of notes used.

References

- Class notes from Data Structures (Unit 1)
- Abdul Bari Youtube - http://www.youtube.com/@abdul_bari